

Cours Complet Apache Spark avec PySpark

Théorie et Application Pratique (Marathon Olist)

Table des matières

1	Introduction à Apache Spark	2
1.1	Qu'est-ce qu'Apache Spark ? 	2
1.2	Pourquoi Spark ?	2
1.3	Concepts Fondamentaux	2
1.4	Écosystème Spark	2
2	Mise en Place de PySpark	3
2.1	Installation	3
2.2	La SparkSession	3
3	DataFrames Spark : Les Bases	3
3.1	Chargement de Données 	3
3.2	Schéma Explicite (Bonne Pratique)	3
3.3	Actions de Base	3
4	Transformations de DataFrames	4
4.0.1	Sélectionner (select)	4
4.0.2	Filtrer (filter, where)	4
4.0.3	Ajouter/Modifier (withColumn)	4
4.0.4	Renommer (withColumnRenamed)	4
4.0.5	Supprimer (drop)	4
4.0.6	Gérer les Nuls (dropna, fillna)	4
4.0.7	Caster (cast)	5
5	Agrégations (groupBy, agg)	5
5.0.1	Grouper et Agréger	5
5.0.2	Pivot (équivalent Tableau Croisé)	5
6	Jointures (join)	5
6.0.1	Syntaxe et Types	5
7	Spark SQL 	5
7.0.1	Utilisation	5
8	User Defined Functions (UDFs) 	6
8.0.1	Définition et Utilisation	6
9	Écriture des Données 	6
9.0.1	Sauvegarder les Résultats	6
10	Application Pratique : Marathon Olist	7
11	Conclusion & Prochaines Étapes	8
11.1	Récapitulatif	8
11.2	Au-delà du Mode Local	9
11.3	Optimisation	9

Introduction à Apache Spark

1.1 Qu'est-ce qu'Apache Spark ?

Apache Spark est un **moteur de calcul distribué open-source**, rapide et généraliste. Il est conçu pour traiter de très grands volumes de données (Big Data) en répartissant les calculs sur un cluster de plusieurs machines.

1.2 Pourquoi Spark ?

- **Vitesse** : Beaucoup plus rapide que son prédécesseur Hadoop MapReduce grâce au traitement en mémoire.
- **Polyvalence** : Supporte divers workloads : traitement batch (Spark SQL), traitement en temps réel (Spark Streaming), Machine Learning (MLlib), et traitement de graphes (GraphX).
- **Facilité d'utilisation** : Offre des APIs de haut niveau en Python (PySpark), Scala, Java et R, avec une syntaxe souvent proche de Pandas pour les DataFrames.
- **Scalabilité** : Conçu nativement pour s'exécuter sur des clusters de centaines, voire milliers de nuds.

1.3 Concepts Fondamentaux

- **RDD (Resilient Distributed Dataset)** : L'API de bas niveau originale de Spark. Collection distribuée et immuable d'objets. Moins utilisée directement aujourd'hui au profit des DataFrames.
- **DataFrame** : L'API structurée de Spark (depuis Spark 1.3). Similaire à un DataFrame Pandas ou une table SQL, mais distribuée et avec des optimisations (Catalyst Optimizer). C'est l'API **principale** en PySpark.
- **Dataset** : Une extension du DataFrame (principalement en Scala/Java) qui ajoute la vérification de type à la compilation. En Python, DataFrame est l'API standard.

C'est le concept le plus important ! Spark ne calcule rien immédiatement lorsque vous écrivez une transformation. Il construit un **plan d'exécution optimisé** (un DAG - Directed Acyclic Graph). Le calcul n'est déclenché que lorsqu'une **action** est demandée.



- **Transformations** : Opérations paresseuses qui créent un nouveau DataFrame (ex : `select`, `filter`, `groupBy`, `join`, `withColumn`).
- **Actions** : Opérations qui déclenchent le calcul et retournent un résultat ou écrivent des données (ex : `show`, `count`, `collect`, `first`, `take`, `save`, `write`).

1.4 Écosystème Spark

Spark n'est pas qu'un simple moteur, c'est une plateforme :

-  **Spark SQL** : Pour interroger des données structurées via SQL ou l'API DataFrame.
-  **Spark Streaming** : Pour traiter des flux de données en temps réel (ou quasi-réel).
-  **MLlib** : Bibliothèque de Machine Learning distribuée.
-  **GraphX** : Pour le traitement de graphes.

Mise en Place de PySpark

2.1 Installation

Le plus simple pour démarrer localement est d'utiliser pip :

```
1 pip install pyspark findspark
```

2.2 La SparkSession

C'est le point d'entrée unique pour interagir avec Spark.

```
1 from pyspark.sql import SparkSession
2 import findspark
3 # findspark.init() # Utile si Spark n'est pas dans le PATH
4
5 spark = SparkSession.builder \
6     .appName("MonAppSpark") \
7     .master("local[*]") \
8     .config("spark.driver.memory", "4g") # Optionnel: Allouer de la RAM
9     .getOrCreate()
10
11 print(f"SparkSession prête (Version: {spark.version})")
12 # La variable 'spark' est votre outil principal
```

DataFrames Spark : Les Bases

3.1 Chargement de Données

Spark lit de nombreux formats. Pour les CSV :

```
1 df = spark.read.csv(
2     "chemin/vers/fichier.csv",
3     header=True,          # La première ligne est l'en-tête
4     inferSchema=True,     # Tente de deviner les types (préférer un schéma explicite
5                             pour la prod)
6     sep=";",              # Séparateur (optionnel si c'est une virgule)
7     encoding="UTF-8"      # Encodage (optionnel)
8 )
```

Autres formats : `spark.read.parquet("...")`, `spark.read.json("...")`, `spark.read.format("jdbc").options(...).load()`

3.2 Schéma Explicite (Bonne Pratique)

Pour éviter les erreurs de type et accélérer le chargement, définissez le schéma :

```
1 from pyspark.sql.types import StructType, StructField, StringType, IntegerType,
2     DoubleType, TimestampType
3
4 schema_explicite = StructType([
5     StructField("id_commande", StringType(), True), # Nom, Type, Nullable?
6     StructField("id_client", StringType(), True),
7     StructField("montant", DoubleType(), True),
8     StructField("date_achat_str", StringType(), True) # Charger comme String d'abord
9 ])
10 df = spark.read.csv("...", header=True, schema=schema_explicite)
11 # Puis convertir la date string en timestamp avec withColumn
```

3.3 Actions de Base

- Afficher les premières lignes : `df.show(5, truncate=False)`
- Afficher le schéma (types) : `df.printSchema()`
- Compter les lignes : `num_lignes = df.count()`

- Récupérer les données dans Python (ATTENTION : seulement pour de petits résultats!) : `liste_python = df.collect()`
- Afficher les colonnes : `print(df.columns)`



Transformations de DataFrames

► Sélectionner (select)

```
1 from pyspark.sql.functions import col
2 df_select = df.select("col1", col("col2"), df.col3)
3 # Différentes syntaxes pour référencer une colonne
4 df_select.show(3)
```

► Filtrer (filter, where)

```
1 df_filtre = df.filter(col("age") > 30)
2 df_filtre2 = df.where("pays = 'France' AND montant < 100")
3 df_filtre.show(3)
```

► Ajouter/Modifier (withColumn)

```
1 from pyspark.sql.functions import to_timestamp, datediff, lit, year, month
2
3 # Calculer une nouvelle colonne
4 df_new = df.withColumn("montant_ttc", col("montant") * 1.2)
5
6 # Convertir un string en date
7 df_dates = df.withColumn("vraie_date", to_timestamp(col("date_str"), 'yyyy-MM-dd'))
8
9 # Calculer une différence de dates
10 df_delai = df_dates.withColumn("delai_jours", datediff(col("date_fin"), col("date_debut")))
11
12 # Ajouter une colonne constante
13 df_const = df.withColumn("source", lit("FichierA"))
14
15 # Extraire année/mois
16 df_ym = df_dates.withColumn("annee", year(col("vraie_date"))) \
17     .withColumn("mois", month(col("vraie_date")))
18 df_ym.show(3)
```

► Renommer (withColumnRenamed)

```
1 df_renomme = df.withColumnRenamed("ancien_nom", "nouveau_nom")
```

► Supprimer (drop)

```
1 df_drop = df.drop("colonne_inutile1", "colonne_inutile2")
```

► Gérer les Nuls (dropna, fillna)

```
1 # Supprimer les lignes avec des nuls dans certaines colonnes
2 df_no_na = df.dropna(subset=["col_essentielle1", "col_essentielle2"])
3
4 # Remplacer les nuls par une valeur (0 pour les chiffres, 'Inconnu' pour le texte)
5 df_fill = df.fillna(0, subset=["col_num1", "col_num2"])
6 df_fill = df_fill.fillna("Inconnu", subset=["col_str1"])
```

► Caster (cast)

```
1 from pyspark.sql.types import IntegerType, DoubleType
2 df_cast = df.withColumn("montant_int", col("montant_str").cast(IntegerType()))
```



Agrégations (groupBy, agg)

► Grouper et Agréger

```
1 from pyspark.sql.functions import sum, avg, count, countDistinct, min, max, first
2
3 # Calculer la somme et la moyenne par catégorie
4 df_agg = df.groupBy("categorie") \
5     .agg(
6         sum("montant").alias("revenu_total"),
7         avg("montant").alias("revenu_moyen"),
8         count("*").alias("nombre_ventes"),
9         countDistinct("id_client").alias("nombre_clients_uniques")
10    ) \
11    .orderBy(col("revenu_total").desc()) # Trier
12
13 df_agg.show()
```

► Pivot (équivalent Tableau Croisé)

```
1 # Exemple: Ventes par catégorie (lignes) et par année (colonnes)
2 df_pivot = df.groupBy("categorie") \
3     .pivot("annee") # La colonne dont les valeurs deviendront les nouvelles colonnes
4     .agg(sum("montant")) \
5     .fillna(0) # Remplacer les nuls après pivot
6
7 df_pivot.show()
```



Jointures (join)

► Syntaxe et Types

```
1 # Syntaxe
2 df_joint = df_commandes.join(
3     df_clients,
4     on="id_client", # Clé commune (si même nom)
5     # Ou: on=df_commandes["cli_id"] == df_clients["cust_id"],
6     how="left"      # Type: inner, left, right, full
7 )
8
9 # Gérer les colonnes ambiguës après jointure
10 df_joint = df_joint.drop(df_clients["id_client"]) # Supprime la colonne dupliquée
11
12 df_joint.show(5)
```



Spark SQL

► Utilisation

```
1 # 1. Enregistrer les DataFrames comme vues temporaires
2 df_commandes.createOrReplaceTempView("commandes_sql")
3 df_clients.createOrReplaceTempView("clients_sql")
4
```

```

5 # 2. Exécuter la requête SQL
6 query = """
7     SELECT
8         c.ville_client,
9         COUNT(o.id_commande) as nb_commandes
10    FROM commandes_sql o
11    JOIN clients_sql c ON o.id_client = c.id_client
12    WHERE o.statut_commande = 'delivered'
13    GROUP BY c.ville_client
14    ORDER BY nb_commandes DESC
15    LIMIT 10
16 """
17 df_resultat_sql = spark.sql(query)
18
19 # C'est un DataFrame Spark normal !
20 df_resultat_sql.show()

```

⚡ User Defined Functions (UDFs) ⚡

► Définition et Utilisation

Quand les fonctions natives ne suffisent pas.

```

1 from pyspark.sql.functions import udf
2 from pyspark.sql.types import StringType
3
4 # 1. Fonction Python
5 def ma_logique_complexe(valeur1, valeur2):
6     if valeur1 > valeur2:
7         return "Supérieur"
8     else:
9         return "Inférieur ou égal"
10
11 # 2. Enregistrement comme UDF
12 ma_logique_udf = udf(ma_logique_complexe, StringType()) # Spécifier le type de retour
13
14 # 3. Application dans withColumn
15 df_avec_udf = df.withColumn(
16     "comparaison",
17     ma_logique_udf(col("col_a"), col("col_b"))
18 )
19 df_avec_udf.show(5)

```

Attention : Préférer les fonctions natives Spark si possible pour la performance.

⚡ Écriture des Données 💾

► Sauvegarder les Résultats

```

1 chemin_sortie = "chemin/vers/dossier_resultat.parquet"
2
3 df_resultat.write \
4     .format("parquet")           # Format (parquet, csv, json, orc...)
5     .mode("overwrite")          # Mode (overwrite, append, ignore, error)
6     .partitionBy("annee", "mois") # Optionnel: Organiser les fichiers par partition
7     .option("compression", "snappy") # Optionnel: Compression pour Parquet/ORC
8     .save(chemin_sortie)        # Lance l'écriture (Action)
9
10 print("Données sauvegardées !")

```

Le format **Parquet** est fortement recommandé pour le stockage Big Data.



Application Pratique : Marathon Olist

</> Reprise du Code du Marathon avec Explications

Cette section reprend le code Python que nous avons écrit, en le commentant à la lumière des concepts Spark vus précédemment.

```
1 # Imports nécessaires (déjà vus)
2 import findspark
3 from pyspark.sql import SparkSession
4 from pyspark.sql.functions import col, to_timestamp, datediff, count,
    countDistinct, udf, lit, sum, avg, year, month
5 from pyspark.sql.types import StringType
6 from pathlib import Path
7
8 # Initialisation de la SparkSession (cf. Section 2)
9 spark = SparkSession.builder \
10     .appName("MarathonPySpark_Olist") \
11     .master("local[*]") \
12     .config("spark.driver.memory", "4g") \
13     .getOrCreate()
14
15 chemin_dataset = Path('/Users/madjiguenedotesene/Desktop/Projet E-commerce')
```

Listing 1 – Imports et Initialisation Spark

```
1 # Chargement des CSV (cf. Section 3)
2 df_commandes_spark = spark.read.csv(str(chemin_dataset / 'olist_orders_dataset.
    csv'), header=True, inferSchema=True)
3 # ... charger tous les autres ...
4 df_produits_spark = spark.read.csv(str(chemin_dataset / 'olist_products_dataset.
    csv'), header=True, inferSchema=True)
5 df_traduction_categorie_spark = spark.read.csv(str(chemin_dataset / '
    product_category_name_translation.csv'), header=True, inferSchema=True)
6
7 # Renommage des colonnes avec withColumnRenamed (cf. Section 4)
8 df_commandes_spark = df_commandes_spark.withColumnRenamed('order_id', '
    id_commande') # ... etc.
9 df_produits_spark = df_produits_spark.withColumnRenamed('product_category_name'
    , 'nom_categorie_produit') # ... etc.
10
11 # Nettoyage des dates avec withColumn et to_timestamp (cf. Section 4)
12 df_commandes_spark = df_commandes_spark.withColumn("date_achat", to_timestamp(
    col("order_purchase_timestamp"))) # Utiliser le nom original avant renommage
    si besoin
13 # ... convertir toutes les colonnes dates ...
14
15 # Nettoyage des Nuls avec fillna (cf. Section 4)
16 df_produits_spark = df_produits_spark.fillna('inconnu', subset=['
    nom_categorie_produit'])
17 df_produits_spark = df_produits_spark.fillna(0, subset=['product_weight_g', '
    product_length_cm']) # Simplifié
```

Listing 2 – Chargement, Renommage, Nettoyage (Heure 1)

```
1 # Création de df_principale_spark avec les jointures (cf. Section 6)
2 df_principale_spark = df_commandes_spark.join(df_articles_commandes_spark, on='
    id_commande', how='inner') # ... joindre les autres ...
3 # ... ajouter les colonnes calculées (temps_livraison, annee_achat) avec
    withColumn ...
4 df_principale_spark = df_principale_spark.withColumn("temps_livraison_jours",
    datediff(col("date_livraison_client"), col("date_achat")))
5 df_principale_spark = df_principale_spark.withColumn("annee_achat", year(col("
    date_achat")))
6
```

```

7 # Nettoyage final post-fusion
8 # ... fillna ...
9
10 # Agrégations avec groupBy et agg (cf. Section 5)
11 # Top Villes
12 top_10_villes = df_principale_spark.groupBy('ville_client') \
13     .agg(countDistinct('id_client_unique').alias('nb_clients')) \
14     .orderBy(col('nb_clients').desc()).limit(10)
15 top_10_villes.show()
16
17 # Agrégation Temporelle (Alternative à Resample)
18 # Ventes par Mois (cf. Section 5)
19 ventes_par_mois = df_principale_spark.groupBy(year("date_achat").alias("annee")
20     , month("date_achat").alias("mois")) \
21     .agg(sum("valeur_paiement").alias("revenu_mensuel")) \
22     .orderBy("annee", "mois")
23 ventes_par_mois.show()
24
25 # Pivot Table (cf. Section 5)
26 pivot_revenu_etat_annee = df_principale_spark.groupBy("etat_client") \
27     .pivot("annee_achat") \
28     .agg(sum("valeur_paiement")) \
29     .fillna(0)
30 pivot_revenu_etat_annee.show()

```

Listing 3 – Jointures et Agrégations (Heure 2 3)

```

1 # Définition de l'UDF (cf. Section 8)
2 def categoriser_livraison(jours):
3     # ... logique ...
4     return categorie
5 categoriser_livraison_udf = udf(categoriser_livraison, StringType())
6
7 # Application de l'UDF
8 df_avec_categorie_spark = df_principale_spark.withColumn(
9     "categorie_livraison", categoriser_livraison_udf(col("temps_livraison_jours")
10 ))
11 df_avec_categorie_spark.select("id_commande", "temps_livraison_jours", "
12     categorie_livraison").show(5)
13
14 # Écriture au format Parquet (cf. Section 9)
15 chemin_sortie_parquet = str(chemin_dataset / "resultats_analyse_spark.parquet")
16 df_avec_categorie_spark.write \
17     .format("parquet") \
18     .mode("overwrite") \
19     .save(chemin_sortie_parquet)
20 print(f"Résultats sauvegardés en Parquet : {chemin_sortie_parquet}")
21
22 # Arrêt de Spark
23 spark.stop()

```

Listing 4 – UDF et Écriture (Heure 4)



Conclusion & Prochaines Étapes

11.1 Récapitulatif

Apache Spark, via son API PySpark, offre un cadre puissant et scalable pour le traitement de données massives. L'API DataFrame, avec sa syntaxe proche de Pandas et la possibilité d'utiliser Spark SQL, rend la transition accessible. L'évaluation paresseuse et l'exécution distribuée sont ses forces majeures.

11.2 Au-delà du Mode Local

Ce cours s'est concentré sur l'exécution locale. Les vraies forces de Spark apparaissent sur un **cluster** :

- **Standalone Cluster** : Simple à mettre en place pour débiter.
- **YARN / Mesos** : Gestionnaires de ressources de cluster (souvent utilisés avec Hadoop).
- **Kubernetes** : Devient un standard pour déployer Spark.
- **Services Cloud Managés** : **AWS EMR**, **Google Dataproc**, **Azure Databricks/HDInsight**. Ils simplifient grandement la gestion du cluster.

11.3 Optimisation

Pour travailler sur de très gros volumes, des techniques d'optimisation sont cruciales :

- **Partitionnement** : Contrôler comment les données sont distribuées sur le cluster.
- **Caching / Persistence** : Garder des DataFrames intermédiaires en mémoire.
- **Broadcast Joins** : Optimiser les jointures avec une petite table.
- **Choix des Formats** : Utiliser Parquet ou ORC.

Fin du Cours

Vous avez maintenant les bases théoriques et pratiques pour commencer à utiliser PySpark pour vos analyses de données, y compris sur de grands volumes. La prochaine étape est de pratiquer sur des projets plus conséquents et d'explorer l'exécution sur cluster.